E-Commerce Purchase-Intent Prediction with Explainable AI

Final project report

Sundram Sharma

1. Summary

This project predicts whether an online shopping session ends in a purchase, explains every prediction, and serves the result through an API and a dashboard. It follows the CRISP-DM structure in the project brief. On a held-out test set of 2,400 sessions the deployed model reaches **PR-AUC 0.558** (95% CI 0.51-0.61) and **ROC-AUC 0.837**, against a purchase base rate of 15.7%.

Three findings matter more than the headline number:

1. **PageValues was excluded on purpose.** No session with PageValues = 0 ever converts (6,612 of 12,000 sessions), and in a live visit the value is only known late. Using it would raise test PR-AUC from 0.56 to 0.85, a deployment-leakage figure.
2. **A one-line baseline is close.** ProductRelated alone gives test PR-AUC 0.531; the full pipeline adds about 0.03.
3. **The operating threshold is a business decision.** Moving the assumed value-to-cost ratio from 2:1 to 40:1 moves the best threshold from 0.47 to 0.02.

2. Verification status

Component	Status
Data loader + validation, preprocessing, feature engineering, metrics	Run; covered by unit tests
Training pipeline (imbalance benchmark, tuning, CV comparison, feature selection, threshold, single test evaluation)	Run end to end (`python -m src.train`)
Algorithms	**4 of the 6 requested were trained** (LogisticRegression, DecisionTree, RandomForest, GradientBoosting). XGBoost, LightGBM and CatBoost could not be installed in the build sandbox. They are registered automatically in `src/models/trainer.py` when installed: run `pip install -r requirements.txt` then `python -m src.train` to get all six.
ML backend	`mini_sklearn` (NumPy fallback implementing the scikit-learn API subset). With scikit-learn installed, the real library is used automatically, and numbers will differ slightly.
Explainability	Model-agnostic permutation-sampling **Shapley values** and a **LIME-style** surrogate implemented in `src/explain/` (verified by tests: efficiency, exactness on an additive model). The `shap` and `lime` packages themselves were not available; hooks for them are documented.
Notebooks (4)	Executed, 0 errors (`tools/build_notebooks.py`)
Unit tests	51 tests (50 pass, 1 skipped: needs pydantic)
FastAPI service (`src/api`)	Written to the FastAPI/Pydantic v2 API, **never started** (packages not installable). The scoring logic it calls is tested.
Streamlit dashboard (`dashboard/`)	Written, **never rendered**. Its data layer is tested.
Docker, Compose, MLflow, GitHub Actions, AWS notes	Written, **not executed**. Experiment runs are logged to `experiments/runs.jsonl` when MLflow is absent.
Code style (black / ruff)	**Not run** (tools unavailable). Some lines exceed 120 characters,

	so the first CI run will likely need `black .` and a few ruff fixes.

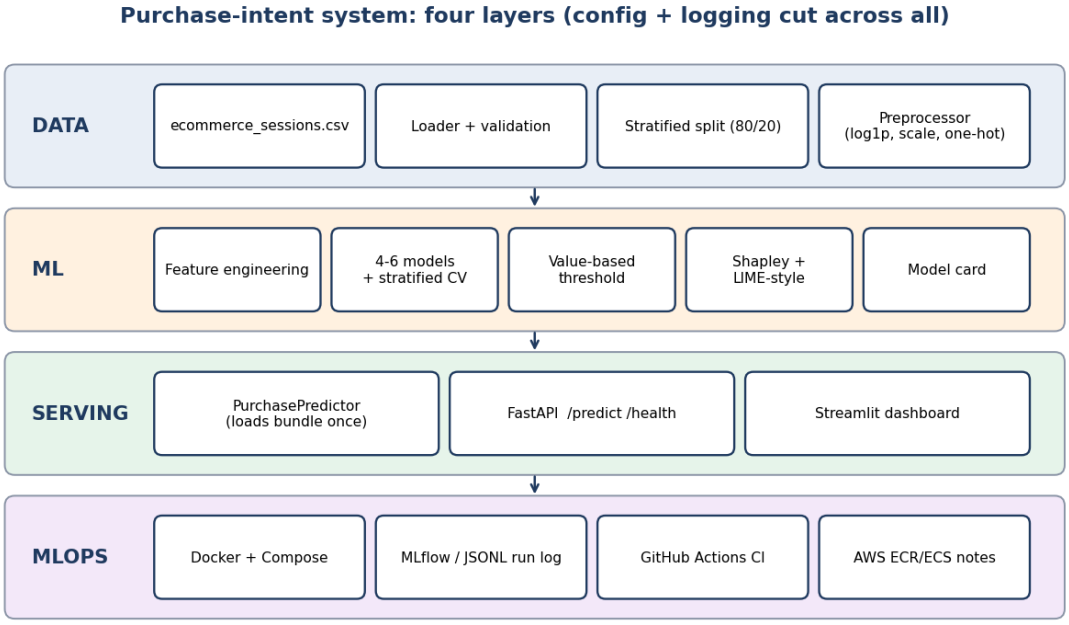
3. Business problem and success criteria

A retailer wants to find sessions likely to convert so that low-cost interventions (a banner, a nudge, a remarketing audience) can be targeted. Because only about one session in six converts, accuracy is misleading: “always no purchase” is 84.3% accurate and finds no buyers. The metrics that lead are therefore PR-AUC and recall on the buying class, with ROC-AUC, F1, Brier score and a confusion matrix as support.

4. Data

12,000 sessions with 17 inputs and the target Converted (15.7% positive). No missing values and no duplicate rows. Four ID columns (OperatingSystems, Browser, Region, TrafficType) are stored as integers but are categories, so they are one-hot encoded and never scaled. Conversion is highest in November (20.0%) and lowest in February and June (about 10%); new visitors convert less (10.2%) than returning visitors (16.6%). The file looks synthetic: 463 sessions have BounceRates above ExitRates, which real analytics data should not show. Results therefore describe this dataset.

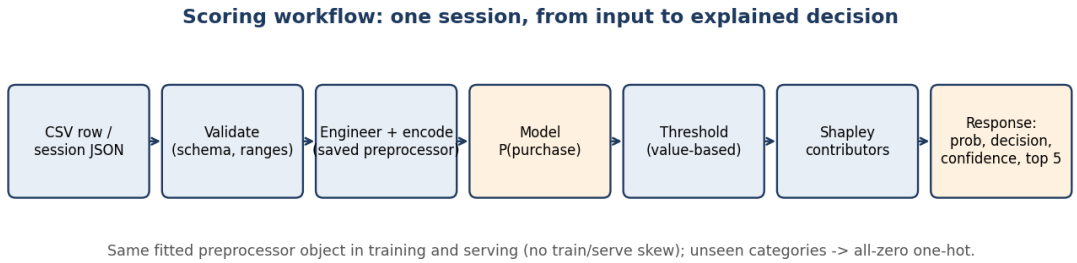
5. Method



Architecture

The data were split 80/20, stratified and seeded (the test part was used once for the final model). Everything that chooses something (imbalance strategy, hyper-parameters, features, threshold) uses the training part only with stratified 5-fold cross-validation. Preprocessing (log1p and standardisation of skewed numerics, one-hot encoding of categoricals with unseen categories mapped to zeros, optional SMOTE) is fitted inside each fold and saved as one object that serving reuses, so there is no train/serve skew.

Imbalance strategies were benchmarked rather than assumed. Plain probabilities with threshold tuning and class weights were within noise of each other for every model, and **SMOTE was never better and often worse** (for example DecisionTree 0.43 against 0.48 CV PR-AUC).



Scoring workflow

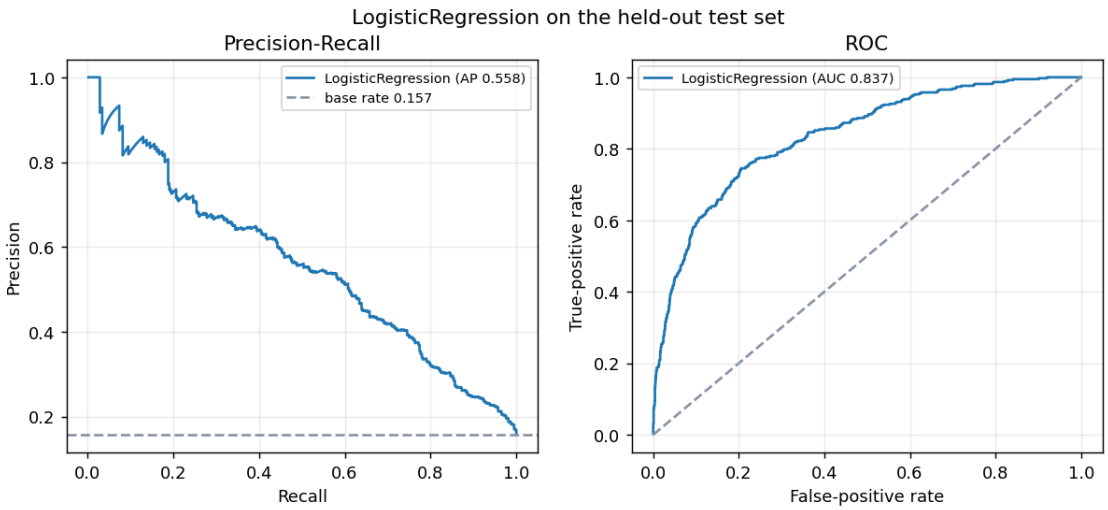
6. Results

Model	Imbalance	CV PR-AUC	CV ROC-AUC	CV Brier	Test PR-AUC	Test ROC-AUC
LogisticRegression	none	0.531 +- 0.028	0.832	0.100	0.555	0.836
DecisionTree	none	0.489 +- 0.020	0.802	0.105	0.479	0.797
RandomForest	none	0.532 +- 0.025	0.830	0.099	0.543	0.832
GradientBoosting	class_weight	0.521 +- 0.021	0.830	0.159	0.553	0.835

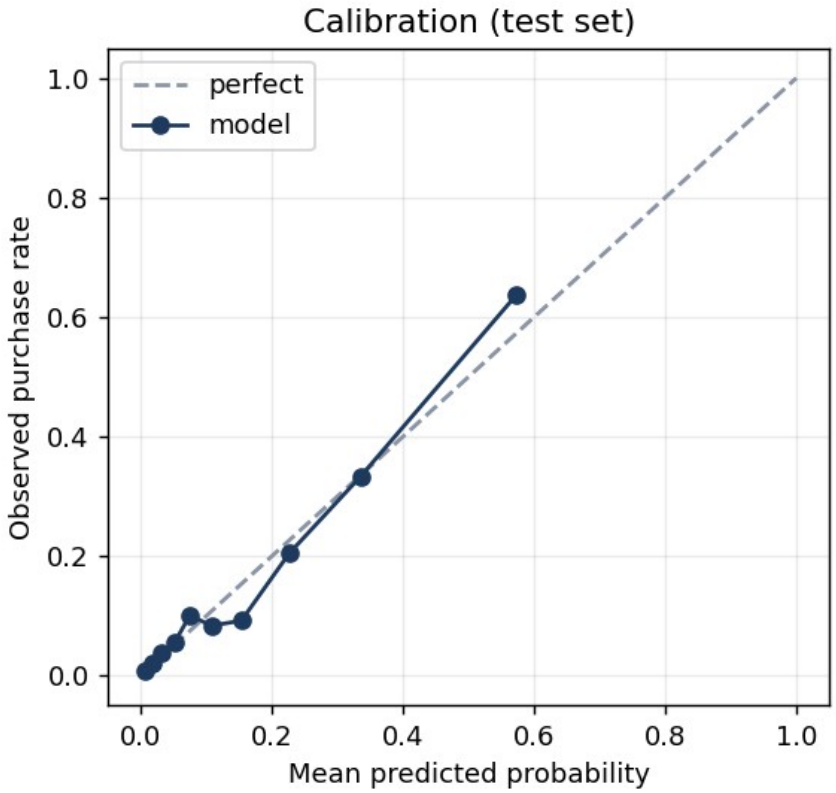
Four of the six requested algorithms were trained; XGBoost, LightGBM and CatBoost are registered automatically when installed. LogisticRegression, RandomForest and GradientBoosting are within one standard error of each other, so the **one-standard-error rule** picked the simplest, **LogisticRegression**. This rule was adopted after the first run showed LogisticRegression and RandomForest tied to three decimals; the raw-best model (RandomForest) has test PR-AUC 0.543, not higher than the deployed model's 0.558. GradientBoosting's Brier score is much worse because class weights distort its probabilities.

Feature selection. Permutation importance (drop in PR-AUC when one input is shuffled) is dominated by ProductRelated, then ExitRates; everything else is at or below noise. Six inputs (TrafficType, Weekend, Month, SpecialDay, OperatingSystems, Region) were removed under a stated rule, with CV PR-AUC 0.531 before and 0.538 after (a difference inside the noise: the claim is no loss with fewer inputs, not an improvement). The engineered ratio features add 0.0006 CV PR-AUC, which is also noise; they are kept for interpretability only.

Test set (used once). PR-AUC 0.558, ROC-AUC 0.837, Brier 0.098. At threshold 0.125: recall 0.81, precision 0.32, TP 306, FP 663, FN 71, TN 1360.



Test PR and ROC curves



Calibration on the test set

7. Decision threshold from business value

Net value of flagging a session = value of a captured purchase x purchases flagged - cost per intervention x sessions flagged. The value (40) and cost (4) are **assumptions**, to be replaced with the retailer's real figures. They give an upper bound because they assume every flagged would-be buyer is captured. The threshold was chosen on training out-of-fold predictions.

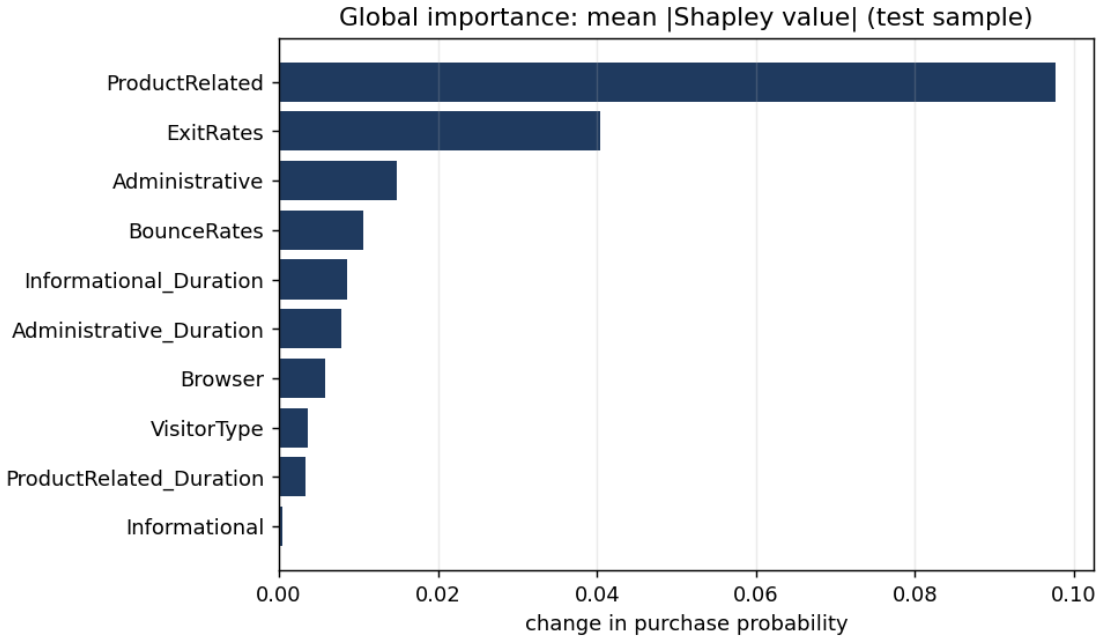
value:cost	threshold	precision	recall	flagged (train OOF)
2:1	0.470	0.65	0.32	744
5:1	0.255	0.47	0.63	2032
10:1	0.125	0.32	0.83	3846

20:1	0.045	0.22	0.95	6445
40:1	0.020	0.19	0.99	8001

On the test set the net value is 8,364, compared with 5,480 for flagging everyone and 0 for flagging nobody. Flagging everyone is a strong baseline when a purchase is worth ten interventions, so the model's practical gain depends on how expensive interventions really are and on whether they change behaviour, which this data cannot show.

8. Explainability

Global importance uses mean absolute Shapley value in probability units (ProductRelated about 10 points, ExitRates about 4, all others 1.5 or less). Its ranking agrees with permutation importance (Spearman 0.89). The Shapley explainer is a model-agnostic permutation-sampling estimator; contributions add up exactly to the prediction minus the baseline, which the unit tests verify, along with exactness on an additive model. A LIME-style local surrogate gives a second view: it names the same top feature in 92% of sessions, but its local fit is weak (mean R2 0.22), so its numbers are a rough second opinion. The shap and lime packages themselves could not be installed; both methods explain the model and say nothing about causes.



Global Shapley importance

Scoring with an explanation takes about 16 ms and scoring alone 7 ms on the build machine (the explainer is built once at start-up).

9. Deployment assets

The repository contains a FastAPI service (/predict, /health, /model-info, validated input, extra fields such as PageValues rejected), a six-section Streamlit dashboard that loads the saved model and never retrains, a Dockerfile, a Compose file (API, dashboard, MLflow UI), a GitHub Actions workflow (lint, format check, tests, image build), MLflow logging with a JSON-

lines fallback, and AWS ECR/ECS notes. **None of these could be executed in the build environment**; they were written to the libraries' documented APIs and should be run and corrected as needed. The scoring logic they wrap is covered by 51 unit tests.

10. Limitations and next steps

- Only whole-session totals exist, so scoring earlier in a visit was never evaluated.
- Two features carry almost all the signal; more behavioural signals (clickstream order, cart events that are available early) would be the highest-value addition.
- The value and cost figures are assumed; an uplift experiment (randomised nudges) is needed to know whether interventions change the outcome.
- Train all six algorithms with the real libraries, and re-check which model wins; add drift monitoring and a calibration check in production.

11. Reproducibility

config/config.yaml holds every tunable value; the seed is 42; the data hash is 14acae89e75a4700. python -m src.train rebuilds all numbers in this report from the CSV, and python tools/make_docs.py regenerates the README, model card and this text from the saved results.